

PromptCanary: Detecting Silent Behavioral Drift in Large Language Model APIs

PromptCanary Contributors

2026

Abstract

Large language model (LLM) providers routinely update the models behind a stable API identifier without announcing the change or exposing a version number a caller can pin to. When such an update alters output formatting, reasoning style, tool-calling structure, or refusal behavior, applications built on top of that API can regress silently: no exception is raised, no status code changes, and the failure surfaces only when a downstream parser breaks or a user notices something is wrong. We present PromptCanary, an open-source Python library and command-line tool that applies the golden-master regression-testing pattern, well established in traditional software engineering, to this problem. A user defines a small suite of representative prompts and a set of scoring functions (“probes”); the tool runs the suite against a provider, saves the result as a versioned baseline, and on subsequent runs reports exactly which probes regressed, by how much, and against which prompt. We describe the system’s design, its probe taxonomy and scoring model, its provider-agnostic architecture built on top of LiteLLM, and its testing methodology, which favors deterministic mock providers and property-based invariant checking over live API calls. We report on a set of independently executed, illustrative case studies — run separately from our own development environment, using the published package — that demonstrate correct end-to-end behavior and surface a genuine design limitation: probes in the current version are scoped to a suite, not to individual prompts, which can dilute the drift signal with persistent, expected non-matches. We discuss this and other limitations candidly, including the heuristic nature of several pattern-matching probes and the absence of statistical drift modeling. PromptCanary is released under the MIT license and is available via the Python Package Index.

1. Introduction

Software regression testing rests on a simple premise: capture a known-good output, and alert whenever a later run produces something different. This pattern — often called golden-master or snapshot testing — has worked well for decades because traditional software is deterministic. Given the same input, a correctly functioning program produces the same output every time, so any difference is, by definition, a regression worth investigating.

Large language models break this premise in two ways. First, their outputs are inherently non-deterministic even at fixed sampling parameters, so “different” cannot simply mean “wrong.” Second, and more specific to this paper’s motivation, the model behind a given API identifier is not fixed over time in the way a compiled binary is. Providers retrain, fine-tune, and swap the model serving a given endpoint on their own schedule, and callers generally have no way to detect this beyond noticing that behavior has changed. A prompt that reliably produced a JSON object with keys in a particular order last month may, without warning, produce the same information with the keys reordered, wrapped in a markdown code fence it previously omitted, or preceded by a conversational preamble it previously did not include. None of these changes raise an error. All of them can break a downstream parser, an agent’s tool-calling loop, or a user-facing feature that depended on the old behavior.

This failure mode — silent behavioral drift — is distinct from the failure modes that existing LLM evaluation tooling is built to catch. General capability benchmarks (measuring, for instance, how well a model answers graduate-level science questions) are designed to compare different models against each other, not to detect when the *same* named endpoint has changed. Correctness-oriented testing frameworks for LLM applications typically check whether a single response meets some criterion, but do not by default persist a baseline and diff future runs against it. What is missing is something narrower and more mundane: a lightweight, git-friendly way to say “here are twenty prompts that matter to my application, here is what a good response used to look like, and please tell me the moment that stops being true.”

PromptCanary is built to fill exactly that gap. It is not a general evaluation harness, a safety classifier, or a benchmark suite, and we are explicit throughout this paper about what it does not attempt to do. Its scope is deliberately narrow: define prompts, define scoring functions, save a baseline, compare future runs against that baseline, and report the result in a form a human or a continuous-integration pipeline can act on immediately.

The remainder of this paper describes the system’s design (Section 3), its implementation and testing methodology (Section 4), a set of independently executed case studies that exercise the published package end to end (Section 5), and a candid discussion of the tool’s current limitations (Section 6), several of which were identified through the very case studies we report rather than through deliberate red-teaming — a distinction we think is worth being transparent about.

2. Related Work and Positioning

Three adjacent bodies of tooling inform PromptCanary’s design without being duplicated by it.

General-purpose LLM evaluation harnesses measure a model’s capability across broad task suites (question answering, reasoning, coding, and similar), typically to compare one model against another or to track progress across model generations. These tools answer “how good is this model,” which is a different question from “has this specific endpoint’s behavior changed since I last checked,” and they are not typically designed to be run against a narrow, user-authored set of production-relevant prompts with a persisted base-

line.

LLM application testing frameworks focus on verifying that a single response, or a single conversation, satisfies some property — for instance, that a chatbot’s reply stays on topic or that a generated SQL query is syntactically valid. These are valuable for correctness testing at development time but generally do not address the drift problem, since they do not by default compare today’s output against a saved record of yesterday’s.

Classical regression and snapshot testing is the most direct intellectual ancestor of this work. The golden-master pattern — save an accepted output, fail the build when a new run differs from it — is decades old and well-understood for deterministic systems. PromptCanary’s contribution is not this pattern itself but its adaptation to a domain where “differs from” cannot be a boolean equality check. A response can be different in inconsequential ways (rephrased sentences that convey the same fact) or in consequential ways (a missing JSON field), and a workable tool needs a scoring layer, not a diff, to tell these apart. This is the role the probe abstraction plays, described in Section 3.3.

PromptCanary sits at the intersection of these three areas: the persistence and diffing discipline of snapshot testing, applied through a graduated, per-property scoring model, to the specific and previously underserved problem of detecting when a provider’s own model has silently changed.

3. System Design

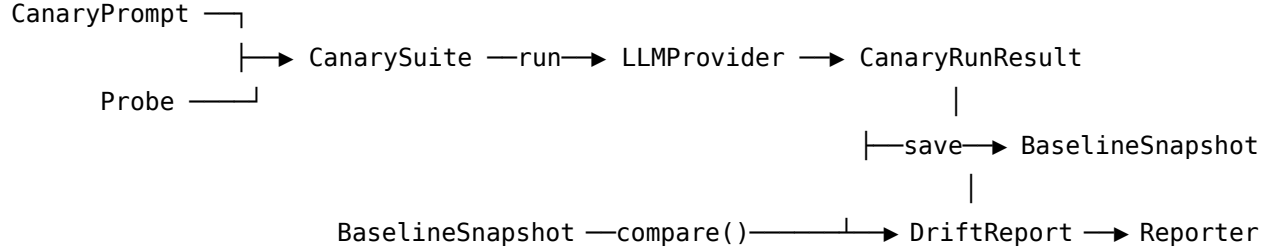
3.1 Design Principles

The system follows five principles, stated here because they explain several design decisions in the sections that follow:

1. **Simplicity first.** Most users should be productive without reading the source. A working suite should be expressible in fewer than thirty lines of YAML.
2. **Pluggable everything.** Providers, scoring probes, and baseline storage are each defined behind an abstract interface, so any one of them can be replaced without touching the other two.
3. **No silent failures in the tool itself.** A user-authored probe that raises an exception during evaluation must not crash the run; it is caught and converted into a failed, clearly labeled result. The tool that exists to catch silent failures elsewhere should not silently fail.
4. **Deterministic by default.** Sampling temperature defaults to zero and a fixed random seed is passed when the provider supports one, to minimize sampling noise in the drift signal.
5. **Reproducible, inspectable storage.** Baselines are plain JSON files, intended to be committed to version control alongside the prompts that produced them, so that a baseline change is reviewable in a pull request like any other code change.

3.2 Core Abstractions

The system’s data flow is organized around six types:



A `CanaryPrompt` pairs a text prompt with optional metadata: an identifier, tags, expected keywords, and an optional per-prompt system-prompt override. A `Probe` is a stateless, callable unit that scores one prompt-response pair and returns a structured result (Section 3.3). A `CanarySuite` bundles a list of prompts with a list of probes and drives the run loop against a provider, producing a `CanaryRunResult`. That result can be persisted as a `BaselineSnapshot` — a versioned, timestamped, git-friendly JSON record — and a later `CanaryRunResult` can be compared against a saved snapshot to produce a `DriftReport`, which is in turn rendered by a `Reporter` into a terminal view, a Markdown document suitable for a pull-request comment, a self-contained HTML page, or raw JSON for downstream automation.

Every one of these types is a strongly-typed, validated data model rather than a loosely structured dictionary, which lets the tool serialize and deserialize baselines without ambiguity and gives users editor-level autocompletion and type checking when scripting against the library directly.

3.3 The Probe Abstraction and Scoring Model

A probe is the unit that turns a raw text response into an actionable signal. Each probe returns two pieces of information: a boolean pass/fail result, used for binary gating decisions such as failing a continuous integration job, and a continuous score between zero and one, used for trend tracking and partial credit.

The separation between these two fields is deliberate and, we think, the single most consequential scoring decision in the system. A purely binary probe cannot distinguish a response that is missing one of five expected JSON keys from a response that is not JSON at all; both would simply fail. A purely continuous score, on the other hand, gives no unambiguous answer to the question a deployment pipeline actually needs answered: is it safe to proceed? By returning both, a probe can report, for instance, that four of five expected keys are present (score = 0.8) while still marking the result as failed because a required field was missing, and downstream tooling can choose which signal to act on for which purpose.

Nineteen built-in probes ship with the library at the time of writing, organized into five categories:

Category	Examples	What it detects
Format & structure	JSON validity, key order, response length, markdown headers, keyword presence	Changes in how output is structured
Reasoning style	Step-by-step reasoning presence, verbosity, hedging language, unsolicited preamble	Changes in how a model explains itself
Safety & refusal	Refusal detection, safety-disclaimer injection	Newly appearing or disappearing refusals and caveats
Tool use	Tool-call presence, correct function name, argument completeness, full schema validation	Regressions in agent/function-calling behavior
Factual & tone	Fixed-answer consistency, lightweight sentiment	Drift on anchor facts or overall tone

Probes are registered automatically: any subclass of the base probe class that defines a non-empty identifier is discovered and made available by name, with no separate registration call. A lighter-weight decorator form is also provided for simple, function-based probes. This mechanism is exercised directly in Section 5.4, where a fresh Python interpreter correctly discovers both the built-in probes and several probes defined inline in a notebook, with no manual wiring.

3.4 Baseline Comparison Semantics

Comparing a new run to a saved baseline is done by matching results on the pair (probe identifier, prompt identifier), not by run order. This means a suite’s prompts or probes can be reordered, or a new probe can be added, without breaking the comparison; a probe or prompt present on only one side of the comparison is treated as an implicit failure on the side where it is missing, so removing a prompt or probe is itself a detectable, reportable event rather than a silent gap.

For each matched pair, the system computes a score delta (current score minus baseline score) and classifies the pair as a regression whenever that delta drops by more than a configurable threshold and the pair no longer passes — whether because a previously passing result stopped passing, or because an already-failing result got measurably worse — with the symmetric rule for classifying an improvement. This means a probe that was already failing before a given change, and fails more severely after it, is still correctly flagged as a regression rather than treated as unchanged simply because it failed on both sides of the comparison. An overall severity label — NONE, LOW, MEDIUM, HIGH, or CRITICAL — is derived from the regression rate and the single worst observed delta. We are explicit that this severity label is a fixed, hand-authored heuristic rather than a statistically fitted or learned model; it has not been calibrated against human judgments of how serious

a given regression is, and users who need a different sensitivity profile can and should adjust the underlying thresholds rather than treat the default labels as authoritative. This heuristic’s behavior, including a case where it correctly escalates to CRITICAL on genuine new regressions despite a noisy baseline, is examined directly in Section 5.3.

3.5 Provider Abstraction

Rather than writing a separate adapter for every LLM API, the system delegates provider communication to LiteLLM, a widely used library that exposes a large number of commercial and self-hosted providers behind one calling convention. This choice keeps the core library’s required dependency surface small while covering the large majority of providers a user might want to test.

A deliberate consequence of this choice is that free, locally hosted, open-weight models served through Ollama are exposed through the exact same interface as commercial APIs, with no code path distinction. We consider this more than a convenience. Because local models have no marginal cost per call, they are the only class of provider for which continuous, high-frequency canary checks (for example, hourly) are economically sensible; commercial providers are better suited to periodic checks (daily or weekly) given per-call cost. A monitoring strategy that layers a zero-cost, high-frequency local check underneath periodic paid-provider checks is, in our view, a genuinely useful pattern this design enables, not merely a byproduct of supporting many providers for its own sake.

For providers LiteLLM does not cover, a two-method abstract base class is provided so a custom adapter can be written without touching any other part of the system.

4. Implementation and Engineering Practices

The library is implemented in Python (3.10 and later) and comprises approximately 5,000 lines of source code, accompanied by roughly 3,450 lines of test code. Domain models are defined with Pydantic, giving run-time validation and JSON serialization for free on every data type described in Section 3.2. The command-line interface is built with Typer and rendered with Rich, and reports are additionally emitted as Markdown, self-contained HTML, and JSON.

Testing philosophy. A deliberate decision was made not to make any real network calls in the automated test suite, and not to rely on cassette-recording tools that replay previously captured HTTP traffic. Instead, every test exercises a deterministic, hand-written mock provider that returns fixed responses. This was chosen for reproducibility (no flakiness from network conditions or provider-side rate limits), cost (zero API spend to run the test suite, including in continuous integration), and speed. The explicit tradeoff, which we record rather than gloss over, is that this approach does not exercise real provider-specific response-shape quirks — for instance, unusual streaming chunk boundaries or provider-specific error payload formats — and a bug specific to how one provider’s client library structures its response objects would not necessarily be caught by this suite. Coverage of the code path that parses real provider responses is instead achieved by mocking the

underlying HTTP call at the point where the provider-abstraction library returns control to our code, which exercises our own parsing and error-handling logic without requiring network access.

Property-based testing. Beyond example-based unit tests, a number of structural invariants are checked with generated, randomized inputs rather than hand-picked examples. Concretely, the test suite mechanically verifies, across randomly generated inputs, that every probe score remains within the closed interval from zero to one; that a run’s overall score is always exactly the arithmetic mean of its constituent probe scores; that a drift report’s overall score delta is always exactly equal to the current score minus the baseline score; that comparing a run against itself never produces a false-positive regression; that a specific format-validity probe is always exactly binary and never returns an intermediate value; and that a saved-and-reloaded baseline is bit-for-bit faithful to the run it was saved from. These are properties that a handful of hand-written examples can suggest but cannot exhaustively confirm, which is precisely the case for which generated-input testing is well suited.

Continuous integration and a note on process. The project’s automated checks run linting, a code-formatting check, static type checking in strict mode, and the full test suite on every change. In the course of preparing this work, two continuous-integration failures were traced to the same underlying cause: a development environment that had, over time, accumulated packages installed for unrelated reasons, which meant that a missing entry in the project’s declared dependency list went unnoticed locally and surfaced only when the automated pipeline performed a genuinely clean install. Both instances were fixed at the level of the dependency declaration rather than by working around the symptom, and the project’s release process now includes an explicit step of installing into a disposable, empty virtual environment before every release, specifically to catch this class of error before it reaches continuous integration rather than after. We mention this not because it is remarkable — this is a common failure mode in any project with an evolving dependency set — but because we think a paper claiming engineering rigor should describe its actual process, including the parts that required correction, rather than only its intended process.

At the time of writing, the full test suite comprises 261 tests using only the project’s required development dependencies (an additional four tests, covering an optional visualization feature, are skipped unless an optional plotting dependency is also installed), passing with 89% line coverage of the source tree.

5. Case Studies

The material in this section comes from three notebooks, executed independently on a hosted, third-party notebook environment separate from our own development machine, using the package as installed from the public package index rather than from a local source checkout. We report this material as a set of illustrative, qualitative case studies, not as a controlled empirical evaluation. No claim of statistical significance is made anywhere in this section; the value of these runs is that they demonstrate correct end-to-end behavior in a genuinely independent environment, using the artifact a real user would actually install, and that in the course of demonstrating the tool, they surfaced a real usability finding we had not anticipated in advance.

5.1 End-to-End Reproducibility

Four notebooks — covering a quickstart walkthrough, a continuous-integration and multi-provider scheduling guide, a gradual drift simulation, and custom probe authoring — were executed in full independently, comprising 80 cells in total, 42 of them executable code, with zero runtime exceptions. This is a modest but meaningful check: it confirms that the packaged, published artifact behaves identically to the source check-out used during development, and that the library’s public interface is usable by code that has never seen the project’s internal source tree, only its documented API.

5.2 Case Study A: A Clean Deployment Gate

The most straightforward case study wraps the library’s comparison logic in a two-outcome deployment gate: run the suite, compare to the last accepted baseline, and permit or block a hypothetical deployment based on the resulting severity label. On a first run with no existing baseline, the tool correctly identified the absence of a baseline and saved the current run as the new reference point rather than failing. On a second run against a deliberately altered mock response, the tool reported a score drop from 100% to 50%, correctly classified the change as CRITICAL severity, and the surrounding logic correctly translated that classification into a blocked deployment decision. Because every probe in this suite was chosen to be semantically relevant to the prompt it scored — a JSON-validity check against a prompt that asked for JSON, a refusal check against a prompt with no reason to be refused — the signal was unambiguous: a 50-percentage-point drop with no other explanation.

5.3 Case Study B: A Gradual Drift Simulation, and What It Revealed

A second notebook simulated eight days of runs against a synthetic provider whose probability of returning a “drifted” response increased on a fixed schedule, from zero on the first two simulated days to a maximum on the final day. Four prompts were each scored by six probes.

The resulting score history did not start at 100% on the clean first day, as we had expected when designing the demonstration; it started at 66.7%, and held at that same value through day 3 of the eight-day simulation (days 0 through 3, spanning drift probabilities of zero, zero, ten, and twenty percent), before dropping to 57.6% on day 4, the first day the injected drift probability reached forty percent. Examining the per-probe breakdown explained why: three of the six probes — a JSON-validity check, a keyword-presence check, and a step-by-step reasoning check — were applied uniformly across all four prompts, but only one prompt in the suite actually asked for JSON, only one asked a question with a specific expected keyword, and only one asked for step-by-step instructions. The other three probes were, in effect, guaranteed to fail on prompts they were never meant to evaluate, for the entire duration of the simulation, including the supposedly clean baseline days.

This is a real design characteristic of the current system, not a bug in the sense of producing an incorrect result: every probe listed in a suite is applied to every prompt in that suite, and the system does not currently provide a way to say “only run this probe against that specific prompt.” We had, in fact, written the demonstration

content without noticing this consequence until seeing the executed output, which we think is itself worth reporting plainly: this finding emerged from actually running our own example material end to end, not from a design review conducted before release.

The more encouraging half of this case study is what the comparison logic did with that noisy baseline. Because drift detection in this system is based on the *change* between a baseline and a current run, not on the absolute score, the three chronically-failing, non-applicable probes contributed no false alarms at all on the days where nothing had actually changed: comparing each subsequent day’s run against the day-0 baseline, the tool reported NONE severity with zero detected regressions for days 1, 2, and 3, despite an absolute score on those days well below 100%. The first CRITICAL classification, with three newly detected regressions, appeared on day 4 — the first day the injected drift probability (40%) was high enough to newly break three previously-passing probes: a step-by-step reasoning check, a response-verbosity check, and an unsolicited-preamble check. This is precisely the change the simulation was designed to eventually produce. In other words, a persistently imperfect baseline degraded the tool’s absolute score without degrading its ability to correctly localize when something genuinely new went wrong.

We treat both halves of this finding as significant. The first is a concrete, near-term design limitation, discussed further in Section 6: the system should allow a probe to be scoped to specific prompts rather than applied suite-wide by default, and this case study is direct evidence for why. The second is a robustness property worth stating plainly, because it was not designed in deliberately and only became visible through this imperfect, honestly-reported demonstration: comparison by delta rather than by absolute threshold gives the system meaningful tolerance for a baseline that is itself imperfect.

5.4 Case Study C: Custom Probe Extension and Partial Scoring

A third notebook defined three custom probes from outside the library’s own source tree — one using the lightweight decorator form, two as full subclasses — and confirmed that all three were automatically discoverable through the library’s probe registry alongside the nineteen built-in probes, without any explicit registration step, in a freshly started interpreter. The registry reported twenty-two total entries: nineteen built in, three defined in the notebook itself.

Two of the custom probes demonstrated graduated, partial-credit scoring with concrete, reproducible numbers. A tool-call argument probe scored a fully correct function call at 1.00, a call missing one of two expected arguments at 0.50, and a call with no function invocation at all at 0.00 — a smooth, monotonic decline rather than a single pass/fail cliff. A sentence-count probe, checking whether a response’s length fell within a tolerance band around an expected value, scored an exactly-matching response at 1.00, responses one sentence above or below the target at 0.80, and a response six sentences beyond the target at 0.00 — again a graduated curve rather than a step function. We highlight these two results specifically because graduated scoring is the mechanism by which this system is intended to surface *gradual* drift, of the kind demonstrated in Section 5.3, before it hardens into an outright failure, and this case study confirms that the mechanism behaves as designed when exercised independently.

6. Limitations

We list the limitations we consider most consequential, several of which are direct, named findings from Section 5 rather than abstract concerns.

Probes are scoped to a suite, not to individual prompts. As demonstrated in Section 5.3, every probe in a suite is currently evaluated against every prompt in that suite. When a suite mixes prompts that call for different kinds of response (a factual question, a JSON-formatted request, a step-by-step explanation), probes relevant to only one of those kinds will register expected, permanent non-matches against the others, diluting the suite’s absolute score even though the comparison logic remains sound. Per-prompt probe scoping is the most immediate design change this work has identified as worth prioritizing.

Several probe categories rely on heuristic pattern matching, not semantic understanding. Detection of refusals, safety disclaimers, and sentiment is implemented with keyword and regular-expression matching rather than a learned classifier. This is fast, dependency-free, and transparent — a user can read exactly which pattern matched — but it is not robust to creative phrasing that does not match a known pattern, and it can register a false positive when ordinary text happens to contain a matching phrase for an unrelated reason. Users relying on these specific probes for anything safety-critical should treat their output as a starting point for manual review, not a final verdict.

The severity heuristic is fixed and hand-tuned, not learned or formally calibrated. The thresholds that map a regression rate and worst-case delta onto a NONE-through-CRITICAL label were chosen by engineering judgment and can be overridden, but they have not been validated against a dataset of human severity judgments, and we make no claim that they generalize well beyond the kinds of suites described in this paper.

The test suite’s use of deterministic mocks, while deliberate (Section 4), means genuine provider-specific response quirks are not exercised in continuous integration. A change in how a specific provider’s client library structures an edge-case response (for instance, a partially completed tool call under a length limit) could in principle slip past this testing strategy until observed in real usage.

Drift detection is a single-point comparison, not a statistical or time-series method. The system compares one run against one saved baseline; it does not model variance across repeated runs, account for seasonality, or apply any formal change-point detection statistics. A result that happens to differ from the baseline due to ordinary sampling variance, rather than a genuine underlying change, is not distinguished from a real regression except by the user’s own choice of temperature and threshold settings.

7. Availability

PromptCanary is released under the MIT license and is distributed via the Python Package Index; it can be installed with a single package-manager command and requires Python 3.10 or later. The source, full test

suite, and documentation are maintained in a public repository alongside a running log of the architectural decisions summarized in Section 3, so that the reasoning behind a given design choice remains inspectable alongside the choice itself.

8. Conclusion

PromptCanary applies a long-established idea in software testing — save a known-good output, and flag the moment a new run stops matching it — to a problem that has become common only recently: large language model providers changing the behavior behind a stable-looking API without notice. Its contribution is not a novel algorithm but a carefully scoped piece of engineering: a small, typed, well-tested library that makes this specific failure mode cheap to detect and easy to act on, whether that means failing a continuous-integration job, opening an issue automatically, or simply giving an engineer a clear answer to “did something change?” the next time a user reports that an AI feature is behaving strangely.

We have tried in this paper to describe the system’s actual behavior, including a limitation — suite-wide rather than prompt-scoped probe application — that we did not anticipate at design time and only observed by running our own demonstration material end to end. We think a tool built to surface uncomfortable truths about a model’s behavior should be willing to do the same about its own.